

Milestone Two: Technical Requirements

Project 2: Integration with Payment Provider & Ecommerce Payment Flows

Team 10: Felix Botero, Prisca Louis, Erik Gijlstra, Mariah Weldon

Purpose of the Document:

This document defines the **technical requirements** for Group 10's Capstone Project, *Integration with Payment Provider & E-commerce Payment Flows*. It explains how the system will be designed and developed to meet the functional requirements outlined in Milestone One.

In this document, group 10 will:

- Describe the technologies, tools, and database structures that will be used.
- Provide design details such as architecture, integration points, and security considerations so development is consistent across the team.
- Show how the project will be tested, deployed, and maintained in a classroom environment.

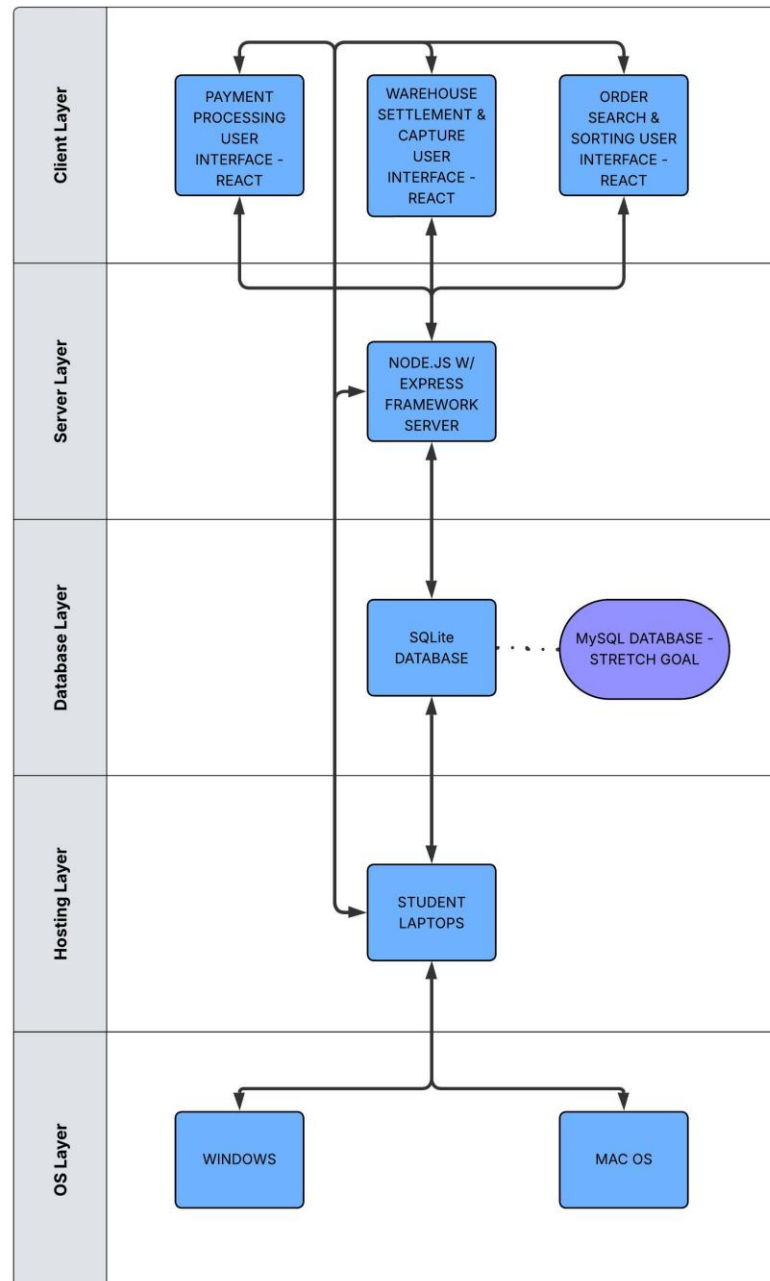
This is a system blueprint that ensures the team shares the same understanding of how the system will be built.

System Architecture:

- **Architecture Type:**
 - Web-based client-server
- **Client Side:**
 - React-based User Interface (JavaScript)
 - Bootstrap Library for styling
- **Server Side:**
 - Node.js
 - Express Framework
- **Database:**
 - SQLite for development

- MySQL stretch goal if time permits
- **Hosting/Environment:**
 - Local development on student laptops
 - Localhost for React & Express
 - SQL based in API folder

Architecture Diagram:



UI HTML/CSS Template Links:

- Checkout UI: [Checkout example for Bootstrap](#)
- Settlement & Database UI: [Examples · Bootstrap v5.3](#) - assembled from various library components

Technology Stack:

- **Programming Languages:**
 - **JavaScript** for both client (React) and server (Node.js/Express)
 - **SQL** (SQLite in dev; MySQL dialect for stretch-goal migration)
- **Frameworks/Libraries:**
 - **Front End:** React 18, Bootstrap 5
 - **Back End:** Express 4
 - **Database Access:** SQLite driver, MySQL driver
- **Database:**
 - **Primary: SQLite 3** - single-file database stored locally
 - **Stretch goal: MySQL 8** - drop-in replacement for production-like hosting
 - **Admin tools (optional):** DB Browser for SQLite; MySQL Workbench
- **Version Control:**
 - Git + GitHub
- **IDE:**
 - **Visual Studio Code** (primary)
 - **IntelliJ IDEA** (alternative/editor for JS & SQL)
 - **Node.js LTS** ($\geq 20.x$), **npm**

Database Design:

ER Design:

Primary Scope Considerations:

- Focus on Functional Requirements
- User Accounts, cart, & login are stretch goals, but not part of the initial development plan
- The payment provider (mock API) will be an external system, not an internal table

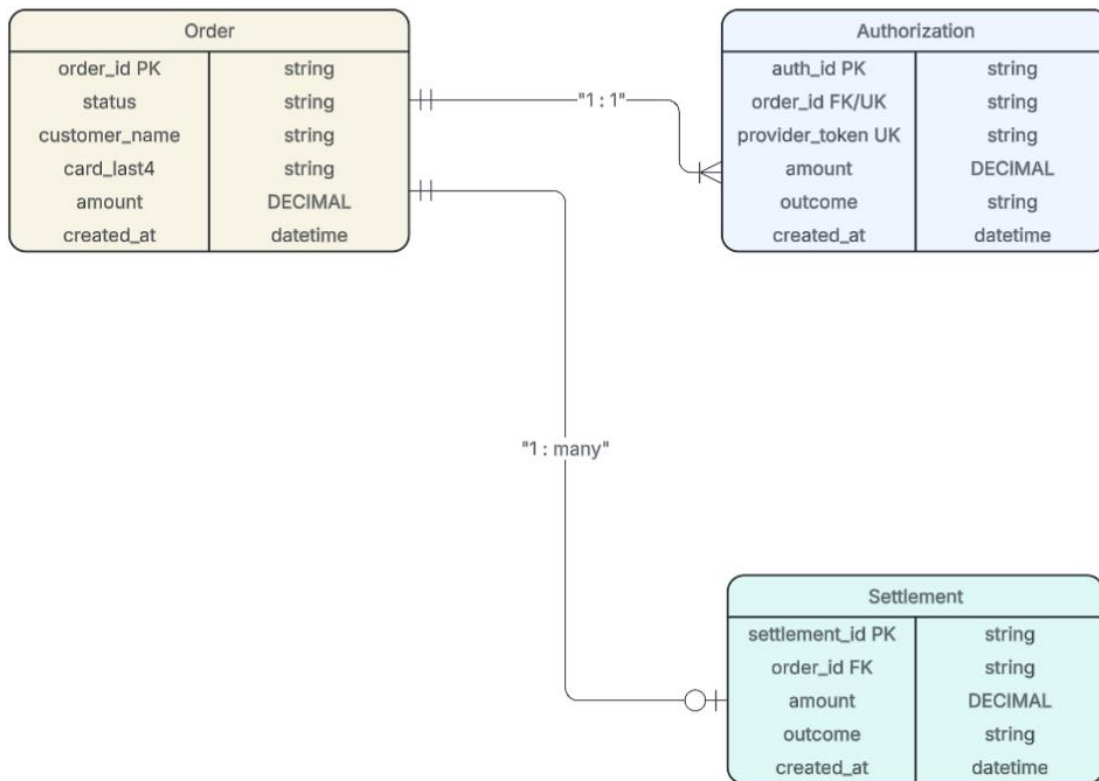
Entities & Attributes:

- Order: *Represents a single checkout submission. Always has a status*
 - order_id – business key shown in UI
 - status – PENDING, AUTHORIZED, SETTLED, ERROR
 - customer_name
 - card_last4
 - created_at
- Authorization: *Stores the result of the mock payment authorization call*
 - order_id – foreign key to order entity; enforces one auth per order
 - provider_token – static token concatenated w/ order_id
 - amount – amount the provider approved/considered
 - outcome – SUCCESS, INSUFFICIENT_FUNDS, INCORRECT_DETAILS, SERVER_ERROR
 - created_at
- Settlement: *Warehouse capture attempts*
 - settlement_id
 - order_id – foreign key to order
 - amount
 - outcome – SUCCESS, EXCEEDS_AUTH
 - created_at

Relationships:

- Order 1 <> 1 Authorization
 - One authorization per order
- Order 1 <> M Settlements
 - An order can have zero or many settlement records
 - Each settlement must not exceed the authorization amount linked to that order

ER Diagram:



Notes & Assumptions:

- Settlement is linked to Order (not to Authorization) so warehouse actions attach to the business object the UI shows (Order). The app can still reference the Order's Authorization to validate amounts.
- Multiple settlements allowed, but each evaluated against the one Authorization
- Customer stored inline on Order (name + last4 only) to avoid adding a customer entity (stretch goal)

Stretch-Goals:

- Allow Authorization Retry: change relationship to Order 1 <> M Authorization and add a "latest" flag or timestamp.
- Add "customer" and "orderlineitem" entities if carts/accounts are added.

Table Structures:

Order

Field	SQLite (dev)	MySQL (stretch)	Notes
order_id (PK)	TEXT	VARCHAR(32)	business key; unique
status	TEXT	ENUM('PENDING','AUTHORIZED','SETTLED','ERROR')	can also use VARCHAR(20) if ENUM not desired
customer_name	TEXT	VARCHAR(120)	
card_last4	TEXT	CHAR(4)	store last 4 only
amount	INTEGER	INT	positive integer
created_at	TEXT (ISO)	DATETIME	default current timestamp

Indexes: UNIQUE(order_id); INDEX(status); INDEX(created_at).

Authorization

Field	SQLite (dev)	MySQL (stretch)	Notes
auth_id (PK)	INTEGER (AI)	BIGINT (AI) or CHAR(36)	surrogate key
order_id (UK, FK)	TEXT	VARCHAR(32)	unique + FK → Order.order_id
provider_token	TEXT	VARCHAR(128)	token = static prefix + order_id
amount	INTEGER	INT	positive integer
outcome	TEXT	ENUM('SUCCESS','INSUFFICIENT_FUNDS','INCORRECT_DETAILS','SERVER_ERROR')	
created_at	TEXT (ISO)	DATETIME	default current timestamp

Indexes: UNIQUE(auth_id); INDEX(outcome); INDEX(created_at).

Settlement

Field	SQLite (dev)	MySQL (stretch)	Notes
-------	--------------	-----------------	-------

settlement_id (PK)	INTEGER (AI)	BIGINT (AI) or CHAR(36)	surrogate key
order_id (FK)	TEXT	VARCHAR(32)	FK → Order.order_id
amount	INTEGER	INT	positive integer
outcome	TEXT	ENUM('SUCCESS', 'EXCEEDS_AUTH')	
created_at	TEXT (ISO)	DATETIME	default current timestamp

Indexes: INDEX(order_id).

Sample Data: Testing Authorization & Settlement Outcomes

Orders

order_id	customer_name	card_last4	amount	status	created_at
ORD-1001	Erik Gijlstra	4242	12.99	AUTHORIZED	2025-09-16T10:00Z
ORD-1002	Prisca Louis	1881	50.00	ERROR	2025-09-16T10:05Z
ORD-1003	Felix Botero	1111	45.00	AUTHORIZED	2025-09-16T10:10Z
ORD-1004	Mariah Weldon	0005	25.99	ERROR	2025-09-16T10:15Z

Authorizations

auth_id	order_id	provider_token	amount	outcome	created_at
1	ORD-1001	STATIC_TOKEN_ORD-1001	12.99	SUCCESS	2025-09-16T10:00Z
2	ORD-1002	STATIC_TOKEN_ORD-1002	50.00	INSUFFICIENT_FUNDS	2025-09-16T10:05Z
3	ORD-1003	STATIC_TOKEN_ORD-1003	45.00	SUCCESS	2025-09-16T10:10Z
4	ORD-1004	STATIC_TOKEN_ORD-1004	25.99	INCORRECT_DETAILS	2025-09-16T10:15Z

Settlements

settlement_id	order_id	amount	outcome	created_at
1	ORD-1001	12.99	SUCCESS	2025-09-16T10:30Z
2	ORD-1003	50.00	EXCEEDS_AUTH	2025-09-16T10:35Z

Test Cases:

- TC-1: Persist SUCCESS authorization (ORD-1001)
 - Steps: Create Order ORD-1001; record Authorization SUCCESS for amount 12.99.
 - Expected: orders.status=AUTHORIZED; authorizations row exists; provider_token=STATIC_TOKEN_ORD-1001.
- TC-2: Persist INSUFFICIENT_FUNDS (ORD-1002)
 - Steps: Create Order ORD-1002; record Authorization INSUFFICIENT_FUNDS for 50.00.
 - Expected: orders.status=ERROR; authorizations.outcome=INSUFFICIENT_FUNDS.
- TC-3: Persist INCORRECT_DETAILS (ORD-1004)
 - Steps: Create Order ORD-1004; record Authorization INCORRECT_DETAILS for 25.99.
 - Expected: orders.status=ERROR; authorizations.outcome=INCORRECT_DETAILS.
- TC-4: Enforce 1:1 Order–Authorization rule
 - Steps: Attempt to add a second Authorization row for ORD-1001.
 - Expected: Operation rejected by database (unique constraint on order_id).
- TC-5: Verify last-4 masking rule
 - Steps: Inspect ORD-1001.card_last4 and ORD-1002.card_last4.
 - Expected: Exactly 4 characters stored (e.g., 4242, 1881); no full card number present.
- TC-6: Settlement SUCCESS within authorized amount (ORD-1001)
 - Steps: Create Settlement for ORD-1001 with amount 12.99.
 - Expected: Settlement outcome=SUCCESS; orders.status updates to SETTLED.
- TC-7: Settlement EXCEEDS_AUTH (ORD-1003)
 - Steps: ORD-1003 has Authorization SUCCESS for 45.00. Create Settlement for 50.00.

- Expected: Settlement outcome=EXCEEDS_AUTH; orders.status remains AUTHORIZED.
- TC-8: Settlement must reference valid Order
 - Steps: Try to create Settlement for non-existent ORD-9999.
 - Expected: Operation rejected by database (FK violation) or API returns error.
- TC-9: Orders list — filter by status
 - Steps: Query orders where status=AUTHORIZED.
 - Expected: Returns only rows marked AUTHORIZED; excludes those with ERROR or SETTLED.
- TC-10: Orders list — sort by created_at (desc)
 - Steps: List all orders sorted newest → oldest.
 - Expected: Order of results matches timestamps (e.g., ORD-1004 newer than ORD-1001).
- TC-11: Authorization token rule
 - Steps: Inspect authorizations.provider_token values.
 - Expected: Format exactly matches STATIC_TOKEN_<order_id>.
- TC-12: Amount field stored and compared in dollars
 - Steps: Verify amounts for ORD-1001 and ORD-1003 across Orders, Authorizations, and Settlements.
 - Expected: Values persist as decimal dollar amounts (e.g., 12.99, 45.00, 50.00) and comparisons use those decimals correctly.

Integration Requirements:

The system integrates with an external mock payment provider API to simulate credit card authorization. This service is provided via a Beeceptor mock endpoint and represents the “Payment Provider” actor identified in the functional requirements.

- External API Endpoint:
 - POST <https://capstoneproject.free.beeceptor.com/authorize>
- Expected Outcomes (randomized by provider):
 - Success response – ~60%
 - Failure: Insufficient Funds – ~17%
 - Failure: Incorrect Card Details – ~17%
 - Error: HTTP 500 Internal Server Error – ~6%
- Role in Project:
 - Called after checkout form submission.
 - Response determines authorization outcome stored in the database.

- Provides realistic test cases for both success and failure without requiring PCI-compliant processing.
- Security/Constraints:
 - Endpoint is mock only; no real payment data processed.
 - Only masked card details (last four digits) are stored in the system after authorization.
- Design Implications:
 - The API is treated as unreliable (random errors); error handling logic is included in the back end
 - Since the API randomizes outcomes, multiple test runs are necessary to observe all scenarios in the actual system.
 - No authentication or API key is required for this endpoint.

Security Requirements:

This project is a simulation only and does not involve real payment processing or PCI compliance. Because of that, the main security focus is on safely handling the mock payment data.

- **Card number masking:** Only the last four digits of a card number will ever be stored or displayed in the system. Full card numbers and CCV codes are not saved in the database or logs.
- **Limited data storage:** Orders will only store the customer's name, last four card digits, and the order amount. Sensitive payment details (full PAN, expiration, CCV) are only used when calling the mock authorization API and are not retained afterward.
- **Error handling:** The system will not show raw error messages from the payment provider to the customer. Instead, it will display a simple error message (e.g., "Authorization failed") while logging the issue for troubleshooting.

Stretch goals: Passwords, logins, and role-based access control

Performance Requirements:

Since this project will run on a student laptop and use a lightweight database (SQLite), performance expectations are modest. The goal is to demonstrate that the payment flow works smoothly during normal use rather than to support large-scale traffic.

- **Authorization & settlement speed:** Payment authorization calls to the mock API and settlement checks in the database should complete within a few seconds.
- **Orders list responsiveness:** Sorting and filtering of stored orders should return results quickly, even as the dataset grows during testing.
- **Local development performance:** The system should run without noticeable lag on a standard laptop with Node.js and SQLite.
- **Scalability (stretch goal):** If deployed to cloud hosting or upgraded to MySQL, the system design should still allow faster processing and support more concurrent requests, though this is outside the primary project scope.

Backup & Recovery:

Because this project is being developed and demonstrated in a classroom environment, the backup plan focuses on protecting source code and project files rather than production data.

- **Individual backups:** Each team member is responsible for maintaining a local copy of their code and project files on their personal computer.
- **Group repository:** All code will be committed and pushed to the shared GitHub repository, which serves as the central backup.
- **Database recovery:** The SQLite database file can be re-created from scratch at any time by re-running the system and generating new test data, so database backups are not critical.

Note: Since the project uses only mock payment data, no sensitive information is stored, and there are no regulatory backup requirements.

Testing & Quality Assurance:

Goal: Verify that the payment flow (checkout → authorization → settlement → listing) works as designed, handles errors gracefully, and stays consistent as the code changes.

Test Types & Tools:

- **Manual API tests:** endpoint calls in VS code
- **Manual UI tests:** Use the web app to submit forms, review lists, and check messages.
- **Database tests:** See *Database Design* section test cases for details.

What We Will Test:

Checkout & Authorization:

- Valid order submits and calls the mock provider.
- Outcome handling shows correct user messages:
 - Success → order becomes **AUTHORIZED**.
 - Insufficient funds / incorrect details → order **ERROR**; message displayed; no crash.
 - Random 500 from provider → friendly error message; app remains usable.
- Card number masking is enforced: only **last four** ever displayed or stored.

Settlement (Warehouse):

- Settlement equal to or below authorized **amount** → **SUCCESS**, order moves to **SETTLED**.
- Settlement above authorized **amount** → **EXCEEDS_AUTH**; order remains **AUTHORIZED**.
- Settlement for non-existent order is rejected with a clear error.

Orders List (Sorting/Filtering):

- Sorting by date shows newest orders first.
- Filtering by **status** shows only matching orders.
- Table remains responsive with several dozen rows.

Error Handling & UX:

- API/network failure shows a simple error (no stack traces).
- Forms show validation messages (missing fields, bad formats).
- Button states/loader (if present) reset properly after errors.

Test Procedures:

API testing:

- POST /authorize with valid payload → expect SUCCESS; verify DB shows AUTHORIZED and token format STATIC_TOKEN_<order_id>.
- POST /authorize with payloads that trigger INSUFFICIENT_FUNDS and INCORRECT_DETAILS outcomes → expect ERROR status on order.

- Simulate provider 500 (try again until it occurs) → friendly client error; server logs record details.
- POST /settlements within authorized amount → SUCCESS; order → SETTLED.
- POST /settlements above authorized amount → EXCEEDS_AUTH; order stays AUTHORIZED.
- POST /settlements for non-existent order → error response.

UI Testing:

- Submit checkout form with valid data → success path.
- Submit with invalid fields (bad month/year/CCV formats) → visible validation messages.
- View Orders page: sort by newest; filter by **AUTHORIZED**; open an order's details.

Acceptance Criteria:

- A typical user can: create an order, receive an authorization result, perform a settlement, and see the order in the list with correct status without errors or confusing messages.
- Masking rule is consistently applied (only last four digits visible/stored).
- Error scenarios do not crash the app; users can retry.

Deployment Requirements:

This project is designed to run in a **local development environment** on student laptops. No production deployment or cloud infrastructure is required for the course.

- **Local setup (primary):**
 - Client (React) runs on <http://localhost:5173>
 - Server (Express/Node.js) runs on <http://localhost:3001>
 - SQLite database file stored locally in the server project folder
 - Start both services with npm run dev commands
- **Version control:**
 - Source code maintained in the shared GitHub repository
 - Each team member can pull, run locally, and contribute via Git

AI Use Disclaimer:

Artificial intelligence tools (OpenAI ChatGPT) were used to assist in drafting parts of this technical requirements document. AI support was limited to **writing guidance, explanations, and example formatting**; all final decisions were made by the project team.

Types of prompts used included:

- Requests for beginner-friendly explanations of technical requirements.
- Help writing the **System Architecture** section (client/server setup, hosting).
- Guidance on the **Technology Stack** (languages, frameworks, database, version control, IDE/tools).
- Step-by-step support for the **Database Design** section (ER diagram, table structures, sample data, test cases).
- Simplified write-ups for **Integration, Security, Performance, Backup, Testing, and Deployment** sections.
- Drafting a concise **Purpose of Document** section.
- Producing this **AI Use Disclaimer** itself.

The AI was not used to generate code for the actual project deliverables. All implementation work, coding, and final editing of this document were completed by the student team.